DE 414: Practical 3 Report

Amy McDermott (26911264)

April 12, 2026

Introduction

This report documents the implementation of a Multinomial Logistic Regression model using Python. The model is trained on the MNIST dataset using a vectorized Softmax function and Minibatch Gradient Descent with the aim to correctly classify digits. This report documents the implementation of the model, the software designed, an optimal hyperparameter analysis, and the results obtained.

Question 1: Softmax Function

Mathematical definition

The Softmax function serves as the activation layer, converting a vector of raw scores (logits) into a categorical probability distribution. For an input vector $\mathbf{z}$ of K classes, the activation function $\sigma(\mathbf{z})_i$ is defined as:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \quad \text{for } i = 1, \ldots, K \tag{1}$$

The resulting output represents a discrete probability distribution over the K classes, where $\sum \sigma(z)_i = 1$ and $\sigma(z)_i \in [0, 1]$.

Software design and algorithmic implementation

The function implementation adopts a vectorized design approach that computes Softmax for an N-dimensional `NumPy` array `x`. The vectorized operations leverage `NumPy` functions that eliminate iterative loops for computational efficiency on large datasets. To ensure numerical stability, the maximum value along the axis is subtracted from the input before exponentiation to prevent overflow/underflow issues. This approach shifts values without altering their relative probabilities. The function then computes the exponentials of the shifted inputs and normalizes them by dividing by the sum of exponentials along the specified axis, resulting in a probability distribution. Listing 1 below shows the implementation of the Softmax function in Python.

```
shift_X = X - np.max(X, axis = axis, keepdims = True) # Subtract the
    maximum for numerical stability
exps = np.exp(shift_X) # Compute the exponentials
return exps / np.sum(exps, axis = axis, keepdims = True) # Compute softmax
```

Listing 1: Softmax function implementation

Question 2: Log-likelihood Function

Mathematical definition

The Log-Likelihood function computes how well predicted probabilities $\hat{\mathbf{y}}$ match the true one-hot encoded labels $\mathbf{y}$, and is thus used to evalute the convergence of the model during training. For N observations and K discrete classes, the log-likelihood $\ell(\mathbf{W})$ is defined as:

$$\ell(\mathbf{W}) = \sum_{n=1}^{N} \sum_{k=1}^{K} y_k^{(n)} \ln\left(\hat{y}_k^{(n)}\right) \tag{2}$$

where $y_k^{(n)}$ denotes the one-hot encoded ground truth and $\hat{y}_k^{(n)}$ represents the predicted probability for class k for sample n.

Software design and algorithmic implementation

The software design computes the log-likelihood for each sample in a batch, ensuring numerical stability by avoiding undefined logarithms. This is achieved by perturbing the predicted probabilities `Y_hat` by a small scalar ϵ (set to 10^{-15}), that bounds the range of the input to the logarithm to $[\epsilon, 1]$ to prevent $\log(0)$, which is undefined and causes numerically undefined behaviour. The function then performs an element-wise logarithm operation to the clipped predictions, yielding log-probabilities for each class per sample. Finally, the log-likelihood is computed by summing the element-wise product of the one-hot encoded targets and the log-probabilities across all samples and classes. This returns an array of log-likelihoods for each sample in the batch. The implementation is shown in listing 2.

```
epsilon = 1e-15 # Perturb Y_hat to prevent log(0) which is undefined
Y_hat = np.clip(Y_hat, epsilon, 1)
log_probs = np.log(Y_hat) # Compute the log of the predicted probabilities
LL = np.sum(Y * log_probs, axis = 1) # Element-wise multiplication of Y
    and log_probs, then sum across classes for each sample
return LL # Log likelihoods per sample
```

Listing 2: Log-likelihood function implementation

Question 3: Extract the minibatch from the training set

Mini-batch training is a common technique in machine learning to balance computational efficiency with convergence stability. Mini-batch training uses a subset of the training data to compute the gradient and update model parameters, leading to more frequent updates per epoch.

Software design and algorithmic implementation

The implemention uses a function `get_mini_batch` to partition the dataset. This function accepts a batch index **i** representing the specified index of the minibatch to be extracted, and a batch size **b** (defaulting to 60). The software design computes the boundaries of the dataset partition by calculating the `start` and `end` indices relative to **i** and **b**, then uses `NumPy` slicing capabilties to extract the corresponding data samples from the feature matrix $\mathbf{X}$ and target vector $\mathbf{Y}$. Listing 3 shows the implementation of the minibatch extraction function in Python.

```
start = i * batch_size # Starting index of the batch
end = start + batch_size # Ending index of the batch
X_mini = X[start:end] # Slicing along first axis (rows) # Slice the data
    matrix X and vector Y to get the minibatch
Y_mini = Y[start:end]
return X_mini, Y_mini
```

Listing 3: Minibatch extraction function implementation

Question 4: Multinomial logistic regression model

Forward pass

A forward pass function is implemented to compute the predicted probabilities `Y_hat` for a given input feature matrix $\mathbf{X}$ and learned weight matrix $\mathbf{W}$. A key engineering design choice includes the augmentation of the input data with a bias term. This is achieved by appending a column of ones to the left of the matrix $\mathbf{X}$, which allows the model to learn an intercept term for each class without requiring a seperate bias vector. The forward pass process consists of two stages:

1. Linear transformation: The augmented input data is multiplied with the weight matrix $\mathbf{W}$ to compute the logits, which are the raw scores for each class.

2. Softmax activation: The logits are passed through the Softmax function to convert them into predicted probabilities **Y_hat** for each class, ensuring that the output is a valid probability distribution.

The implementation of the forward pass function in Python is shown in listing 4.

```
# Add bias term to input data (column of 1s to X matrix)
ones = np.ones((X.shape[0], 1)) # Create a column of ones with the same
    number of rows as X
X_bias = np.hstack([ones, X]) # Append the column of ones to the left of X
logits = X_bias @ self.weights
Y_hat = softmax(logits, axis = 1)
return Y_hat
```

Listing 4: Forward pass function implementation

Gradient descent optimisation

The model parameters $\mathbf{W}$ are optimised using an iterative gradient descent approach. A minibatch protocol is used so as to balance computational efficiency with convergence stability. This optimisation process iterates for a specified number of epochs, where each epoch represents a complete pass through the dataset.

The objective of the optimisation is to maximise the average log-likelihood $\ell(\mathbf{W})$. For a single observation $x^{(n)}$, the partial derivative of the log-likelihood with respect to the weight vector w_k (weight associated with class k) is given by:

$$\frac{\partial \ell^{(n)}}{\partial \mathbf{w}_k} = \left(y_k^{(n)} - \hat{y}_k^{(n)}\right) \mathbf{x}^{(n)} \tag{3}$$

To compute the update for the entire weight matrix $\mathbf{W}$ across a batch of $\mathbf{N}$ samples, the following expression is used:

$$\nabla_{\mathbf{W}}\ell = \frac{1}{N}\mathbf{X}^T\left(\mathbf{Y} - \hat{\mathbf{Y}}\right) \tag{4}$$

Using the calculated gradient, the weights are updated in the direction that maximises the log-likelihood, as specified by the following update rule:

$$\mathbf{W}_{\text{new}} = \mathbf{W}_{\text{old}} + \eta\nabla_{\mathbf{W}}\ell \tag{5}$$

where η is the learning rate (step size).

This optimisation process is implemented in as shown in listing 5 below, where the model weights are updated iteratively for each minibatch across all epochs.

```
num_samples = train_X.shape[0]
num_batches = num_samples // batch_size

for epoch in range(epochs):
    for i in range(num_batches):
        x_batch, y_batch = get_mini_batch(train_X, train_Y, i, batch_size)

        y_hat = self.forward(x_batch) # Forward pass to get predictions for the batch

        ones = np.ones((x_batch.shape[0], 1)) # Create a column of ones with the same number of rows as x_batch
        x_batch_bias = np.hstack([ones, x_batch]) # Append the column of ones to the left of x_batch

        # Calculate the gradient of the log-likelihood wrt. to the weights
        grad = (x_batch_bias.T @ (y_batch - y_hat)) / batch_size # Average over the batch

        self.weights += learning_rate * grad
```

Listing 5: Gradient descent function implementation

Train model

A MultinomialLogisticRegression model is instantiated and is then trained on the MNIST feature matrix and one-hot encoded target vector by calling the `train` method. The hyperparameters used for training include a batch size of 60, an epoch count of 32, and a learning rate of 0.01. The Python implementation is shown in listing 6 below.

```
lr = MultinomialLogisticRegression(train_X.shape[1], train_Y.shape[1])
lr.train(train_X, train_Y, test_X, test_Y, epochs=32, learning_rate=0.01, batch_size=60)
```

Listing 6: Model training implementation

Performance evaluation

To analyse the performance and generalisation capabilities of the trained model, iterative logging of the training and test set log-likelihoods and accuracies is implemented after each epoch of training

to ensure the persistence of the results. The log file is created in a "log" directory, and the results are appended after each epoch of training. After executing the training process, the structured log file can be analyzed to evaluate the model's learning trajectory and assists in identifying any possible overfitting or underfitting. The trends observed provide insights into how well the model generalizes to unseen data and whether it is converging towards a good solution. Listing 7 shows the implementation of the append only file logging functionality and Table 1 summarizes the log file data for the last five epochs of training.

```
with open(log_file, "a") as f:
    f.write(f"{epoch+1},{train_ll},{test_ll},{train_acc},{test_acc}\n")
```

Listing 7: Logging implementation for performance evaluation

Table 1: Model Performance (batch size = 60, epoch count = 32, learning rate = 0.01)

Epoch	**Train LL**	**Test LL**	**Train Acc**	**Test Acc**
28	−0.3019	−0.2935	0.9163	0.9193
29	−0.3007	−0.2926	0.9165	0.9194
30	−0.2995	−0.2918	0.9169	0.9195
31	−0.2984	−0.2910	0.9174	0.9196
32	−0.2974	−0.2902	0.9176	0.9198

The results in the log file demonstrate that the training and test set log-likelihoods are monotonically increasing (becoming less negative) over epochs, which validates that the Gradient Ascent algorithm is successfully maximising the objective function. Furthermore, it can be observed that both the training and test accuracies are increasing over epochs, indicating that the model is improving its performance on both the training and unseen test data. The final test set accuracy of 91.98% suggests that the model is performing well in this specific MNIST digit classification task, correclty classifying a high proportion of the test samples.

Question 5: Hyperparameter search

To identify the optimal hyperparameters for training the Multinomial Logistic Regression model, an exploratory search is conducted over a range of batch sizes and epoch counts. The batch size determines the number of samples utilised in a single iteration to update the model weights; smaller batches provide more frequent updates that can lead to faster convergence, whereas larger batches offer more stable weight updates but often require more epochs to converge to an optimal solution. The number of epochs defines how many time the entire training dataset is passed through during model training. With too few epochs, underfitting will occur, where the model has insufficient iterations to minimise the log-likelhood. With too many epochs, overfitting may occur, where the model learns to fit the training data too closely and fails to generalise to unseen instances of data. Thus, the optimal number of epochs is required so as to ensure the underlying distribution of the data is captured effectively.

The batch size values that are tested include 16, 64, 256, and 1024. This follows a logarithmic scale that covers small, medium, and large batch sizes. The epoch counts tested include 10, 50, and 100. During the training the each model with the varying combinations of batch size and epoch counts, the training and test set accuracies are logged to CSV files for analysis. The results are summarized in the table 2 below.

Table 2: Performance Comparison across Batch Sizes and Epochs

Epochs	**Batch Size**	**Train Acc**	**Test LL**	**Test Acc**
10	16	0.9178	−0.2875	0.9194
50	16	0.9287	−0.2680	0.9245
100	16	0.9322	−0.2655	0.9261
10	64	0.9030	−0.3347	0.9091
50	64	0.9208	−0.2823	0.9214
100	64	0.9256	−0.2726	0.9234
10	256	0.8804	−0.4471	0.8880
50	256	0.9063	−0.3239	0.9115
100	256	0.9144	−0.2984	0.9181
10	1024	0.8458	−0.7277	0.8550
50	1024	0.8846	−0.4225	0.8917
100	1024	0.8969	−0.3630	0.9031

It can be observed that the smaller batch size of 16 reaches higher accuracies in fewer epochs compared to a larger batch size of 1024. This can be attributed to the fact that smaller batches provide a higher frequency of weight updates, which leads to the model converging rapidly to the optimal solution. In contrast, larger batches require more iterations (epochs) to reach a comparable state of convergence, as they provide less frequent updates to the model weights. Thus, it is evident that smaller batch sizes (16) outperform larger batches given the same number of epochs. The model achieving the best performance in terms of test accuracy has a batch size of 16 and an epoch count of 100, yielding a test accuracy of 0.9261.

To prevent data loss, a robust logging strategy is employed that writes the model training metrics to a uniquely named CSV file for each hyperparameter configuration. By defining the file path as `log_file = f"log/log_b{batch_size}_e{epochs}.csv"`, the logging system ensures that results from different model training runs are stored seperately, and do not overwrite each other. Furthermore, the performance metrics - such as log-likelihood and accuracy - are logged throughout the training process instead of only at the end, allowing for monitoring of the model's learning progress and detection of issues such as overfitting or underfitting. This provides insight into the model's convergence behaviour and assists with selecting optimal hyperparameters for training. Despite the 100 epoch training, with a batch size of 16, yielding a test accuracy of 0.9261, further analysis of the logged data reveals that the true optimal hyperparameters for training the model are a batch size of 16 and an epoch count of 98, yielding a test accuracy of 0.9264. This decline suggests that the model begins to overfit, where the model begins to memorise the training data, at the expense of generalisation. Consequently, the optimal hyperparametes for training the model include a batch size of 16 and an epoch count of 98, yielding a test accuracy of 0.9264.

Conclusion